

UNIVERSIDADE FEDERAL DE ALFENAS

Curso de Ciência da Computação — Instituto de Ciências Exatas

Disciplina: DCE673 - Programação Funcional

Simulado P2 — baseado na Lista de Exercícios 02 (List Comprehension, Recursão, Alta Ordem, Lambda)

Nível e formato inspirados nas provas P1 (2025/2026): questões encadeadas (uma função usa a anterior), funções pequenas e objetivas, mistura de recursão pura, list comprehension e alta ordem.

ENUNCIADO DO SIMULADO

1. (20 pt) Faça, em Haskell, **utilizando list comprehension**, uma solução que receba um `Int n` e retorne uma `[Int]` com todos os divisores próprios de `n` (excluindo o próprio `n`).

```
divisores :: Int -> [Int]
```

2. (20 pt) Considerando `divisores`, faça, em Haskell, uma solução que receba uma `[Int] ns` e retorne uma `[(Int,Bool)]` que informa, para cada elemento `x` de `ns`, se `x` é um número perfeito (soma dos divisores próprios é igual a ele mesmo).

```
ehPerfeito :: [Int] -> [(Int,Bool)]
```

3. (20 pt) Considerando `ehPerfeito`, faça, em Haskell, uma solução que receba uma `[Int] ns` e retorne uma `([Int],[Int])`, isto é, uma dupla em que o primeiro elemento é a lista dos números perfeitos contidos em `ns`, e o segundo é a lista dos que não são perfeitos.

```
separaPerfeitos :: [Int] -> ([Int],[Int])
```

4. (20 pt) Faça, em Haskell, **utilizando funções de alta ordem (map/filter) e/ou lambda**, uma solução que receba uma `[String] palavras` e um `Int k`, retornando apenas os tamanhos das palavras de tamanho **maior ou igual a k** (ou seja, retorna a `[Int]` com os tamanhos, não as strings).

```
tamanhosGrandes :: [String] -> Int -> [Int]
```

5. (15 pt) Faça, em Haskell, **usando foldl ou foldr**, uma solução que receba uma `[Int] xs` e retorne um único `Int`, que é o produto de todos os elementos **pares** de `xs` (se não houver elemento par, retorne 1).

```
produtoPares :: [Int] -> Int
```

Boa Prova!

GABARITO COMENTADO

Questão 1 — divisores (20 pt)

Raciocínio: “divisor próprio de n ” é todo x entre 1 e $n-1$ tal que n é divisível por x (resto da divisão = 0). É o mesmo padrão da função `fatores` usada no Exercício 4 da Lista 2 (números perfeitos).

```
divisores :: Int -> [Int]
divisores n = [x | x <- [1..n-1], n `mod` x == 0]
```

Como ler a comprehension: - $x \leftarrow [1..n-1] \rightarrow$ o **gerador**: percorre todos os candidatos de 1 até $n-1$ (não incluímos n , pois queremos divisores *próprios*). - $n \text{ `mod` } x == 0 \rightarrow$ a ****guarda**** (filtro): só mantém os x que dividem n exatamente (resto zero). - O elemento antes do $|$ (x') é o que vai para a lista resultado — aqui não há transformação, só filtragem.

Teste mental:

```
divisores 28 = [x | x <- [1..27], 28 `mod` x == 0]
              = [1,2,4,7,14]
```

Questão 2 — ehPerfeito (20 pt)

Raciocínio: Para cada x da lista `ns`, queremos formar o par $(x, x \text{ é perfeito?})$. “É perfeito” significa que a soma de divisores x é igual a x . Como o enunciado diz “considerando divisores”, devemos **reaproveitar** essa função — assim como nas provas anteriores (`f2` reaproveitando `f_in`).

```
ehPerfeito :: [Int] -> [(Int,Bool)]
ehPerfeito ns = [(x, sum (divisores x) == x) | x <- ns]
```

Detalhamento: - O gerador $x \leftarrow ns$ percorre cada número da lista de entrada.
- Para cada x , construímos a **tupla** $(x, \text{condição})$. - `sum (divisores x) == x` é a expressão booleana: soma os divisores próprios de x e compara com o próprio x .

Versão alternativa por recursão pura (caso a prova pedisse “sem list comprehension”):

```
ehPerfeito :: [Int] -> [(Int,Bool)]
ehPerfeito [] = []
ehPerfeito (x:xs) = (x, sum (divisores x) == x) : ehPerfeito xs
```

Teste mental:

```
ehPerfeito [6,10,28] = [(6,True),(10,False),(28,True)]
```

Questão 3 — separaPerfeitos (20 pt)

Raciocínio: Padrão “separa” (igual ao Exercício 18 da Lista 2 — separar pares e ímpares), mas o critério agora é “é perfeito” em vez de “é par”. O enunciado pede para reaproveitar `ehPerfeito`, então: 1. Gerar a lista de pares (`Int, Bool`) com `ehPerfeito`. 2. Usar list comprehension para filtrar quem tem `Bool == True` e quem tem `Bool == False`, extraindo só o `Int` de cada par.

```
separaPerfeitos :: [Int] -> ([Int],[Int])
separaPerfeitos ns = ([x | (x,True)  <- pares],
                     [x | (x,False) <- pares])
  where pares = ehPerfeito ns
```

Detalhamento (ponto importante — casamento de padrão dentro do gerador): - `where pares = ehPerfeito ns` calcula uma única vez a lista de pares (`Int, Bool`), evitando reprocessar `ehPerfeito` duas vezes. - `[x | (x,True) <- pares]` é uma list comprehension cujo **gerador já faz casamento de padrão**: só os elementos cujo segundo componente da tupla é exatamente `True` “casam” com o padrão `(x,True)` — os demais são descartados automaticamente, sem precisar de guarda separada. - Analogamente para `False`.

Versão alternativa, mais explícita, usando guardas comuns em vez de casar `True/False` no padrão:

```
separaPerfeitos :: [Int] -> ([Int],[Int])
separaPerfeitos ns = ([x | (x,b) <- pares, b],
                     [x | (x,b) <- pares, not b])
  where pares = ehPerfeito ns
```

Teste mental:

```
separaPerfeitos [6,10,28,100] = ([6,28],[10,100])
```

Questão 4 — tamanhosGrandes (20 pt)

Raciocínio: O enunciado pede explicitamente alta ordem (`map/filter/lambda`). A estrutura é a do Exercício 9 da Lista 2:

```
[f x | x <- xs, p x]    map f (filter p xs)
```

Aqui, `p` é o predicado “tamanho da palavra `k`” e `f` é a função “tamanho da palavra” (`length`).

```
tamanhosGrandes :: [String] -> Int -> [Int]
tamanhosGrandes palavras k = map length (filter (\p -> length p >= k) palavras)
```

Detalhamento, passo a passo: 1. `filter (\p -> length p >= k) palavras` — percorre `palavras` e mantém só as strings cujo tamanho é `k`. O `lambda \p -> length p >= k` é o predicado anônimo (poderíamos ter

nomeado uma função separada, mas o lambda evita “poluir” o código com uma função de uso único, como visto na Aula 06). 2. `map length (...)` — aplica `length` a cada string que sobrou no filtro, transformando a lista de strings filtrada em uma lista de inteiros (os tamanhos).

Versão equivalente usando list comprehension (a equivalência é o ponto-chave do conteúdo, então vale citar ambas em prova):

```
tamanhosGrandes :: [String] -> Int -> [Int]
tamanhosGrandes palavras k = [length p | p <- palavras, length p >= k]
```

Teste mental:

```
tamanhosGrandes ["ola","haskell","top","funcional"] 4
= map length (filter (\p -> length p >= 4) ["ola","haskell","top","funcional"])
= map length ["haskell","funcional"]
= [7,9]
```

Questão 5 — produtoPares (15 pt)

Raciocínio: O enunciado pede explicitamente `fold`. Estratégia clássica: filtrar os pares e depois aplicar um `fold` multiplicativo (combina os conceitos de `filter` e `fold` da Aula 05). Mostro também uma versão em `fold` único.

Opção A — separando filtro + fold (mais clara, reaproveita a ideia do `product` da Aula 05):

```
produtoPares :: [Int] -> Int
produtoPares xs = foldr (*) 1 (filter even xs)
```

- `filter even xs` mantém só os números pares.
- `foldr (*) 1 (...)` multiplica todos os elementos da lista filtrada, começando do valor neutro 1 (se a lista de pares for vazia, o resultado é 1, exatamente como pedido no enunciado).

Simulação:

```
produtoPares [1,2,3,4,5,6]
= foldr (*) 1 (filter even [1,2,3,4,5,6])
= foldr (*) 1 [2,4,6]
= 2 * (4 * (6 * 1))
= 2 * (4 * 6)
= 2 * 24
= 48
```

Opção B — fold único, com lambda decidindo a condição a cada passo (sem `filter`):

```
produtoPares :: [Int] -> Int
produtoPares xs = foldr (\x acc -> if even x then x * acc else acc) 1 xs
```

- A função do fold recebe o elemento atual `x` e o acumulador `acc`; se `x` for par, ele entra na multiplicação (`x * acc`); senão, é ignorado (`acc` passa inalterado).
- Essa versão é mais “enxuta” (uma só passada na lista), mas a Opção A é mais legível e reaproveita `filter + fold` separadamente.

Caso a lista não tenha nenhum par:

`produtoPares [1,3,5] = foldr (*) 1 (filter even [1,3,5]) = foldr (*) 1 [] = 1`

Confirma a regra do enunciado.

Observações gerais de estudo (padrões que se repetem nas provas da disciplina)

1. **Encadeamento de funções:** quase toda prova tem uma sequência de questões em que a função `N` reaproveita a função `N-1`. Sempre que o enunciado disser “considerando `f`”, a melhor estratégia é chamar a função anterior dentro da nova, em vez de reescrever a lógica do zero.
2. **List comprehension com casamento de padrão no gerador** (como em `[x | (x,True) <- pares]`) é um truque muito usado para “filtrar e extrair” ao mesmo tempo — combina dois temas (`comprehension + padrões`) numa linha só, e costuma aparecer em prova.
3. **Equivalência comprehension alta ordem:** pratique transformar qualquer `[f x | x <- xs, p x]` em `map f (filter p xs)` e vice-versa.
4. **foldr vs foldl:** para operações associativas e comutativas (soma, produto, `&&`, `||`) o resultado final é o mesmo nas duas, mas a ordem de avaliação muda. Em provas, normalmente qualquer uma das duas é aceita quando a operação é desse tipo.
5. **Casos de borda:** sempre confira o que a função deve retornar para entrada vazia ou “sem nenhum elemento que satisfaça a condição” — nas provas anteriores isso aparece explicitamente (ex.: “se `n` for zero, retorna `\"\"`” na Lista 2; aqui, “se não houver elemento par, retorne 1”).